

The Knowledge Behind the System

A discipline-by-discipline justification of every component in the HealthForecast AI platform — the data science that makes the forecasts honest, the software engineering that makes the system maintainable, and the industrial engineering that turns predictions into operational decisions. Written to train others.

Data Science

Why these models, these validations, these uncertainty bands

Software Engineering

Why this architecture, these patterns, this deployment

Industrial Engineering

Why these optimizations, these policies, these KPIs

Jonathan Lukwichi · JLW Analytics · healthforecast.jlwanalytics.com

Companion to the MSc dissertation and the live platform · Generated from the actual codebase · July 2026

§ Contents & how to teach from this document

CH.	TITLE	DOMINANT LENS
1	The system in one idea: forecast-to-decision	All three
◇	The platform end-to-end — flow diagram	Map
2	Data layer — provenance, privacy, and the single source of truth	SE + DS
3	Preparation — joins, cleaning, and the anatomy of leakage	DS
4	Forecasting — the statistics and the machine learning, honestly compared	DS
5	Validation — why backtesting is the only honest scoreboard	DS
6	The analytics pipeline — plug-in architecture as an engineering philosophy	SE
7	Workforce optimization — labour law as mathematics	IE
8	Inventory optimization — safety stock, (s,S), and the value of a forecast	IE
9	Simulation — Monte-Carlo as the laboratory of operations	IE + DS
10	The AI analyst — a governed agent, engineered for trust	SE + DS
11	The interface — visualization and interaction as engineering	SE
12	Shipping — reproducibility, deployment, and operating within limits	SE
13	Deep dive: the frontend stack — languages, technologies, UI/UX design	SE + UX
14	Deep dive: the backend stack & API design	SE
15	The decision register — every major choice, its alternatives, its justification	All three
16	A training curriculum — six sessions to teach this system	Pedagogy

How to use this as a trainer

Each chapter follows the same skeleton: **what the component does** → **the theory that justifies it** → **why the alternatives were rejected** → **what a student should be able to answer afterwards**. The colored boxes carry the discipline-specific reasoning: **DATA SCIENCE** **SOFTWARE ENGINEERING** **INDUSTRIAL ENGINEERING**. Chapter 13 condenses every design decision into a single table — that table alone is a complete oral-exam preparation. Chapter 14 turns the whole document into a course.

01 The system in one idea: forecast-to-decision

Most analytics projects stop at a prediction. This system's thesis — in both the academic and commercial sense — is that **a forecast has no value until it changes a decision, and the value of forecast accuracy can be measured in money**. Every layer exists to carry information one step along this chain:

raw data → clean features → forecast with uncertainty → optimized decision → explained recommendation

This is the **Forecast-to-Decision stack**. It is not specific to hospitals: any operation with variable demand and costly resources (fisheries, retail, energy, call centres) fits the same five layers.

DATA SCIENCE **Why "uncertainty" is a first-class citizen.** A point forecast ("74 patients") is a guess; a calibrated interval ("74, likely 55–99") is a planning instrument. Every downstream layer consumes the interval, not the point: the roster covers demand at a chosen service level, and safety stock is literally a function of forecast error dispersion. Teaching rule: *a forecast without an interval is not a deliverable*.

SOFTWARE ENGINEERING **Why layers.** Each layer talks only to its neighbours through narrow interfaces (a DataFrame contract, a JSON schema). This is classic *separation of concerns*: the optimizer does not know GitHub exists; the frontend does not know statsmodels exists. Consequence: any layer can be replaced — a new model, a new database, a new UI — without touching the others. That property was proven in practice: the supply engine was completely rewritten (optimization.py → optimization_engine.py) without the frontend changing.

INDUSTRIAL ENGINEERING **Why "against a naive baseline, in money".** IE evaluates interventions by comparing systems, not admiring components. Every optimization page shows the forecast-driven plan *versus* the naive plan (flat staffing, fixed ordering) and prices the gap in Rand. This is the "value of information" concept from decision analysis made visible to a manager.

◇ The platform end-to-end — flow diagram

One request's journey, top to bottom. Solid boxes are components you built; dashed boxes are external services. Read it twice: once following a **forecast** (left path), once following an **AI question** (right path).



Two example journeys to teach with: **(1) Forecast:** browser → POST /api/forecast/run → router pulls G1 from the registry → forecasting.py trains & predicts → result cached at /last → JSON → day cards render. **(2) AI question:** browser → POST /api/ai/chat (stream) → tool loop asks get_forecast → which reads /last (the very numbers on screen) → Claude writes the grounded answer → tokens stream to the chat bubble.

02 Data layer — provenance, privacy, and the single source of truth

What it is. Seven CSVs (daily/hourly arrivals, daily/hourly clinical breakdowns, engineered calendar features, daily/hourly Pretoria weather) live in a private GitHub repository. The backend fetches them server-side with a scoped token (`data_source.py`) and holds them *only in process memory* (`registry.py`) — never on disk.

SOFTWARE ENGINEERING **The Twelve-Factor principle: config in the environment.** Anything that differs between machines — tokens, keys, repo names — lives in environment variables (`.env` locally, the host's Variables screen in production), never in code. Justification: code is shared and versioned; secrets must be neither. The project lived the failure mode: the same app on two machines with two diverging `.env` files produced the "GitHub rejected the token (401)" incident — the fix was procedural (regenerate, re-paste), not code.

SOFTWARE ENGINEERING **Least privilege.** The token is *fine-grained*: read-only, one repository. If it leaks, the blast radius is "someone can read seven CSVs", not "someone can push code". The same principle reappears in the AI layer (read-only tools) — it is the single most transferable security idea in the system.

DATA SCIENCE **Provenance and reproducibility.** One canonical data repo means every environment computes on identical bytes; a dataset version is a git commit. This is data-science reproducibility applied with software tools — the same discipline journals now demand of published analyses.

DATA SCIENCE **The in-memory privacy model — a deliberate trade.** Holding data only in RAM means a restart loses it (it must re-fetch) and memory is a scarce resource (the 512MB incident). In exchange: no data at rest on a cloud disk, minimal governance surface for sensitive health data. The trade was chosen consciously; Chapter 13 records when to revisit it (a real multi-user deployment → PostgreSQL behind the same registry interface).

What a student must be able to answer

- Why is the token never sent to the browser? (Server-side fetch: secrets stay behind the API boundary.)
- What breaks if two copies of the app exist? (Nothing in git; everything in `.env` — secrets do not sync by design.)
- Why CSVs in git rather than a database? (Versioned provenance, zero infrastructure, adequate scale; a database earns its complexity only with concurrent writers or persistence needs.)

03 Preparation — joins, cleaning, and the anatomy of leakage

What it is. `joins.py` merges the raw files into four analysis groups — G1 (daily total), G2 (hourly), G3 (daily by specialty), G4 (hourly clinical) — each one grain, one key, one audited cleaning pass (empty columns dropped, duplicate keys logged, COVID regimes labelled, zero-days flagged).

DATA SCIENCE Why "one table per question". Models consume a design matrix: rows = observations at one grain, columns = features aligned to the same timestamp. Mixing grains (daily targets with hourly features) is the classic silent error. Building each grain once, centrally, means every model, chart and analyzer literally reads the same numbers — eliminating the "two analysts, two answers" failure of ad-hoc merging.

DATA SCIENCE Leakage — the cardinal sin this layer prevents. Leakage = information from the future (or from the answer) contaminating the features. The design prevents its three common forms: **(1) temporal leakage** — backtests slice strictly *before* the cutoff date at the series level, so lag/rolling features can never see the future; **(2) target leakage** — calendar and weather features are exogenous, known in advance by nature; **(3) preprocessing leakage** — cleaning parameters are computed inside the training window during validation, not on the full series. Teaching rule: *every impressive accuracy number should first be suspected of leakage.*

DATA SCIENCE Structural breaks: the COVID regimes. The series contains regime shifts (lockdowns collapsed ED arrivals). Labelling regimes explicitly — rather than deleting the period or ignoring it — lets analyzers condition on them and lets the modeller justify the 730-day training cap: recent data reflects the current regime; ancient data can actively mislead a model about today.

SOFTWARE ENGINEERING Audited transformations. Every join records what it dropped and why. This is the engineering counterpart of a lab notebook: when a column vanishes or rows halve, the audit answers "was that intended?" without re-running anything.

What a student must be able to answer

- Give three examples of leakage and identify which defence in this system blocks each.
- Why cap training at 730 days rather than use all history? (Regime relevance beats sample size once the process has structurally changed.)

04 Forecasting — the statistics and the machine learning, honestly compared

What it is. Two deliberately different engines forecast the same series. The app calls them "the best statistical model" and "the best ML model"; underneath they are SARIMAX and Gradient Boosting.

The statistical engine — SARIMAX

The pipeline grid-searches ARIMA orders $(p,1,q)$ for $p,q \in \{0,1,2\}$ by AIC, then refits the winner as SARIMAX with seasonal order $(1,1,0,7)$.

DATA SCIENCE **The reasoning chain.** **Differencing ($d=1$):** ED arrivals drift over years; modelling day-to-day changes makes the series approximately stationary, which the ARMA machinery requires. **Weekly seasonal term ($s=7$):** the dominant cycle in arrivals is the week (weekend effect) — the seasonal difference and AR term encode "this Monday resembles last Monday". **AIC for order selection:** $AIC = 2k - 2\ln(L)$ penalizes parameters, protecting against overfitting the order to noise; it is the standard, defensible criterion when comparing non-nested ARIMA candidates on the same data. **Prediction intervals:** come from the model's own error variance — statistically principled *if* the model's assumptions (roughly Gaussian, stable variance) hold, which backtesting checks empirically.

The ML engine — Gradient Boosting

`GradientBoostingRegressor` (200 trees, depth 4, learning rate 0.05) on ten engineered features: day-of-week, month, weekend flag, lags 1/2/3/7, rolling 7-day mean & std, rolling 14-day mean.

DATA SCIENCE **The reasoning chain.** **Why boosting:** trees capture non-linear interactions ARIMA cannot express — e.g. "Mondays after long weekends spike, but only outside COVID regimes". Boosting builds trees sequentially on residuals, giving strong accuracy on tabular data with modest tuning; depth 4 and a small learning rate are the standard bias-variance compromise. **Why these features:** lags 1–3 carry momentum; lag 7 carries the weekly rhythm; rolling means carry the local level (what differencing does for ARIMA); rolling std lets the model sense volatile periods; calendar features inject known-in-advance structure. This is the general recipe of *feature-based time-series ML*: turn a forecasting problem into a supervised regression problem. **Recursive multi-step:** to predict day $t+2$ the model feeds its own $t+1$ prediction back as a lag. Trade-off (an exam favourite): recursive strategy trains *one* model but compounds errors along the horizon; the *direct* strategy trains one model per horizon step, avoiding compounding but multiplying training cost and shrinking each model's effective sample. Recursive was chosen for parsimony, and the widening intervals honestly display its compounding error. **Empirical intervals:** boosting emits no variance, so the band is the 2.5th–97.5th percentile of its own validation residuals, widened ~6% per step ahead. This is a non-parametric, assumption-light interval — cruder than SARIMAX's but honest about what is actually known.

DATA SCIENCE **Why two engines (champion–challenger).** No single family dominates all series. Running a statistical and an ML engine side-by-side (a) provides a falsifiable comparison — accuracy is settled by backtest, not fashion; (b) hedges regime change — ARIMA-type models often degrade more gracefully; (c) builds stakeholder trust — a manager can see the "classic" and the "modern" model agree. This mirrors the M-competitions' central finding: comparisons, not claims.

SOFTWARE ENGINEERING **Non-determinism as a systems problem.** Retraining on each run means two runs differ slightly. That is statistically benign but *operationally* confusing — the AI assistant quoting a different number than the screen was a real bug. The fix — cache the last run server-side (`/api/forecast/last`) and make every consumer read it — teaches a general lesson: *when a computation is stochastic, systems that reference it must share one materialized result.*

What a student must be able to answer

- Derive why $d=1$ and $s=7$ fit this series; what plot would justify each? (Trend in the level plot; weekly spike in the autocorrelation.)
- Explain recursive vs direct multi-step and the error-compounding argument.
- Why can the ML band be wider than the SARIMAX band yet more trustworthy? (It is measured from real errors, not assumed from a model.)

05 Validation — why backtesting is the only honest scoreboard

DATA SCIENCE **Random cross-validation is invalid here.** Classic k-fold CV shuffles rows — for time series that trains on the future to predict the past, inflating accuracy. The valid scheme is the **rolling-origin backtest** the app implements: pick a cutoff, train strictly before it, forecast forward, compare with reality, repeat. The UI exposes this to managers ("Backtest a past date"), which is rare and pedagogically powerful: users can audit the model themselves.

DATA SCIENCE **Metric literacy: MAE vs MAPE.** MAE (mean absolute error) is in patients — interpretable, robust. MAPE (percentage) enables cross-series comparison but explodes when actuals approach zero — exactly why the app falls back to interval-tightness for low-volume specialties instead of quoting a meaningless 400% MAPE. Teaching rule: *choose the metric after looking at the denominator*. The window-total error is also reported because planners decide on weekly totals, where daily noise partially cancels — the metric mirrors the decision.

DATA SCIENCE **Calibration of intervals.** A 95% band is *calibrated* if ~95% of actuals fall inside it over many backtests. Accuracy and calibration are separate virtues: a model can have great MAE and lying intervals. The empirical-residual method ties the band to observed errors, making calibration approximately self-enforcing.

SOFTWARE ENGINEERING **The same philosophy applied to code.** The Playwright suite is backtesting for the UI: 112 machine-checked assertions (no overflow, no overlapping text, at 8 viewports) replace "it looks fine on my phone". One rule unifies the whole project: **a claim is worth exactly the strength of the test that checks it** — for models, backtests; for code, automated tests; for deployment, health checks.

06 The analytics pipeline — plug-in architecture as an engineering philosophy

What it is. The Explore dashboard is generated by a pipeline: `GroupProfile` maps semantic **roles** ("the date column", "the target", "the weekend flag") onto actual columns; each analysis is a small **Analyzer class** declaring the roles it needs; the pipeline runs all registered analyzers, isolates failures, and emits typed `Finding` and `Metric` records the UI renders generically. 8 finding analyzers, 19 KPI metrics, 10 section builders.

SOFTWARE ENGINEERING **The Open–Closed Principle, made concrete.** "Open for extension, closed for modification": adding an analysis means *adding one file*; no existing file changes. This is the single most valuable architecture idea in the codebase to teach, because the payoff is visible: the same afternoon that added the busy-day-threshold analyzer touched no other module.

SOFTWARE ENGINEERING **Dependency inversion via roles.** Analyzers depend on an abstraction ("a column playing the target role"), not on a concretion ("total_daily_arrivals"). Consequence: point the profile at a fisheries dataset and every analyzer runs unchanged. This is how you make analytics code portable across clients — the commercial value of the pattern.

SOFTWARE ENGINEERING **Bulkheads: partial failure beats total failure.** Each analyzer runs in a try/except; one crash becomes one missing card plus a logged failure, not a blank page. Borrowed from resilience engineering (circuit breakers, bulkheads) and applied at dashboard scale.

DATA SCIENCE **Findings as structured claims.** Each finding carries headline / mechanism / recommended action / category (risk-watch-stable-trend). Forcing analytics into that schema imposes analytical discipline: no chart ships without an interpretation and an implied decision — EDA in service of operations, not decoration.

07 Workforce optimization — labour law as mathematics

What it is. `optimization_engine.py` builds a binary integer program over the real 23-nurse pool and solves it with CBC (via PuLP).

$$\begin{aligned} & \text{minimize } \Sigma \text{cost}(\text{locum hours}) + M \cdot \Sigma \text{unfilled}(\text{day, shift}) \\ & \text{subject to } x[n,d,s] \in \{0,1\}; \Sigma_{d,s \in \text{week}} \text{hours} \cdot x \leq 45 \text{ per nurse}; x[n,d,\text{Night}] + x[n,d+1,\text{Day}] \leq 1; \Sigma_s x[n,d,s] \leq 1; \text{coverage}(d,s) \geq \text{demand}(d,s) \end{aligned}$$

INDUSTRIAL ENGINEERING **Why integer programming and not a heuristic.** Rostering is combinatorial: 23 nurses $\times$ 30 days $\times$ 3 shifts $\approx$ 2000 binary decisions with interlocking constraints. Greedy or manual methods produce feasible-looking rosters that hide violations (the simulation's own status quo shows 57.9h weeks — unlawful). An IP makes the constraint set *explicit and provable*: if the solver says infeasible-without-locums, that is a mathematical fact you can take to management, not an opinion. CBC (branch-and-bound over LP relaxations) solves this size in seconds — exactness is affordable, so heuristics are unjustified.

INDUSTRIAL ENGINEERING **Modelling craft in the formulation. Law as hard constraints** (45h BCEA cap, 11h rest encoded as "no Day after Night", one shift/day) — these are non-negotiable, so they bind. **Preferences as soft terms** (skill mix, fairness) — desirable, so they carry weights instead. **The big-M slack on coverage** converts "infeasible" into "feasible with priced shortfall": the solver then *reports* the shortfall as locum hours needed — turning a solver artifact into the headline management number. Teaching rule: *the art of OR is deciding what is a constraint and what is a cost.*

INDUSTRIAL ENGINEERING **Demand disaggregation.** The forecast is daily; shifts need per-shift demand. National NDoH nurse-to-patient ratios split the day by observed arrival shares. This "top-down disaggregation by empirical shares" is standard capacity-planning practice when the finer grain lacks its own reliable model.

DATA SCIENCE **The forecast is the demand input.** Staffing to the forecast (rather than to the peak or the mean) is what saves money: peak-staffing wastes quiet days; mean-staffing understaffs busy ones. The six-strategy comparison quantifies precisely this — the value of the forecast is the payroll gap between "forecast-driven lawful" and the naive strategies.

08 Inventory optimization — safety stock, (s,S), and the value of a forecast

What it is. For each of 30 real items (consumables and drugs with real lead times, prices, ordering costs): a reorder point s and an order-up-to level S , evaluated by simulation.

$$s^* = \bar{d} \cdot L + z_{\alpha} \cdot \sigma_D \cdot \sqrt{L} \quad (\text{demand during lead time} + \text{safety stock})$$

INDUSTRIAL ENGINEERING **Reading the formula like an engineer.** $\bar{d} \cdot L$: expected consumption while a replenishment order is in transit. z_{α} : the standard-normal quantile for the chosen service level (95% $\rightarrow$ 1.645) — a *policy dial*, chosen by management, trading stockout risk against holding cost. $\sigma_D \cdot \sqrt{L}$: uncertainty of demand over the lead time (variance adds over independent days, hence the square root). The strategic punchline of the entire thesis sits in σ : **with a forecast, σ is the forecast's residual error, not raw demand variability — a more accurate forecast directly shrinks safety stock**, and therefore working capital. This is the cleanest monetary expression of forecast value in the whole system.

INDUSTRIAL ENGINEERING **Why (s,S) and not EOQ or fixed schedules.** EOQ optimizes order *size* under constant demand — hospital demand is stochastic, so EOQ alone understocks the tails. Periodic fixed ordering ignores current stock. (s,S) is the canonical stochastic-demand policy: order up to S whenever stock falls to s — near-optimal under random demand with fixed ordering costs, simple enough for a storeroom to execute. The policy ladder (naive $\rightarrow$ s,Q $\rightarrow$ R,S $\rightarrow$ static s,S $\rightarrow$ dynamic forecast-driven base-stock $\rightarrow$ oracle) exists to *show* this progression empirically rather than assert it; the lead-time sweep exposes the honest crossover where a 7-day forecast horizon stops covering a long lead time and simpler continuous-review wins — a genuinely non-obvious result the comparison surfaced.

INDUSTRIAL ENGINEERING **The four-cost taxonomy.** Ordering (fixed per order), holding (capital + storage per unit-day), stockout (clinical/financial penalty), wastage (expiry). Every policy is scored on the *sum* — optimizing any single cost in isolation (the intuitive mistake) simply shifts spend into the others. ABC classification (A-items $\approx$ high value/criticality) prioritizes management attention, which is why the UI leads with A-item risks.

09 Simulation — Monte-Carlo as the laboratory of operations

INDUSTRIAL ENGINEERING **Why simulate at all.** The interacting randomness of demand, lead times and expiry makes closed-form cost expressions intractable beyond textbook cases. Monte-Carlo replaces algebra with experimentation: generate 800 plausible 90-day futures, run the candidate policy through each, average the costs. The law of large numbers guarantees convergence; 800×30 seeds gives tight confidence intervals on every KPI (the $\pm$ CI columns in the simulation CSVs).

INDUSTRIAL ENGINEERING **Variance reduction: common random numbers.** When comparing policies A and B, both are run on the *same* random demand streams. Differences then reflect the policies, not luck — the paired-comparison idea from experimental design applied to simulation. This is why the policy ladder's rankings are stable despite stochastic inputs.

DATA SCIENCE **Distributional realism.** Arrivals are simulated as negative binomial, not Poisson — hospital counts are over-dispersed (variance > mean), and Poisson would understate tail risk, silently under-provisioning safety stock and staffing. Lead times draw from a gamma distribution (positive, right-skewed). Choosing distributions to match observed dispersion is the difference between a simulation and an animation.

DATA SCIENCE **Synthetic forecasts calibrated to real error.** The staff-strategy comparison needs forecasts inside the simulation; they are synthesized to match the empirically measured MAPE ($\approx 12.6\%$ from Chapter 6). This closes the loop honestly: the simulated "value of the forecast" is the value of *this system's actual accuracy*, not an idealized oracle.

10 The AI analyst — a governed agent, engineered for trust

What it is. A Claude-powered assistant (`/api/ai/`) that answers questions over live app data through a **tool loop**: the model receives five read-only tool definitions, requests the ones it needs, the backend executes them and returns results, and the model composes a grounded answer (max 4 rounds), streamed to the browser.

SOFTWARE ENGINEERING **The agent pattern with least privilege.** The assistant can *read* forecasts, stock, staffing, plans — it cannot mutate anything, by construction (no writing tools exist). Governance is architectural, not promissory: the safest permission is the one that is impossible to misuse. Every exchange is also written to a durable audit log surfaced on the Admin page — inspectability is a feature, not an afterthought.

DATA SCIENCE **Grounding against hallucination.** LLMs generate plausible text; unconstrained, they invent numbers. Three defences: (1) the system prompt forbids stating figures not returned by a tool; (2) numeric questions *must* route through tools that return real JSON; (3) the forecast tool reads the **same materialized run the screen shows** (`/last`), eliminating the model-vs-screen contradiction users actually caught. Teaching rule: *an LLM in a data product is an interface to computations, never the computation itself.*

DATA SCIENCE **Retrieval without a vector database.** The 12 knowledge cards are retrieved by keyword overlap — deliberately. At 12 documents, embeddings add infrastructure, latency and failure modes for zero retrieval gain. The engineering judgment "the simplest mechanism that meets the requirement" is itself the lesson; the upgrade path (embeddings at hundreds of cards) is known and deferred.

SOFTWARE ENGINEERING **Prompt-as-policy.** The system prompt encodes product decisions: brevity ("lead with the decision"), tone, the accuracy-figure ban for public users, model-naming rules. Prompts are versioned in git like code, because they *are* behaviourally load-bearing code — and were iterated exactly like code (the over-long supply answer → the decision-first rewrite).

11 The interface — visualization and interaction as engineering

SOFTWARE ENGINEERING **Declarative components.** React's model — $UI = f(state)$ — replaces manual DOM bookkeeping: change the state, the screen follows. Pages compose reusable components (KPI cards, 23 chart types, panels); the design system (CSS custom-property tokens, fluid `clamp()` type/space scales) keeps hundreds of screens visually coherent from one definition point.

DATA SCIENCE **Visualization principles embedded in the charts.** Uncertainty is always drawn (bands around forecasts) because a line alone overstates knowledge. Day cards encode demand tiers by color *and* number (pre-attentive + precise). The responsive chart work encodes a Tufte-ish rule: when space shrinks, **reduce chrome, never legibility** — ticks thin out, value labels hide, the data survives. Axis text renders 1:1 via `ResizeObserver` instead of being geometrically squashed — measurement over stretching.

SOFTWARE ENGINEERING **Mobile-first as constraint discipline.** The remediation followed a phased method — audit before fixing, tokens before components, container queries so components respond to *their own* width, then machine verification (112 assertions). The general lesson: responsive design is not sprinkling media queries; it is choosing layout primitives that reflow intrinsically and proving the result at every width.

12 Shipping — reproducibility, deployment, and operating within limits

SOFTWARE ENGINEERING **Docker: the environment as code.** "Works on my machine" is an environment problem; the Dockerfile makes the environment itself versioned and reproducible — Node builds the frontend, Python assembles the API, the submodule pins the exact thesis artefacts, and the identical image runs anywhere. Continuous deployment (push → build → deploy on Render) shrinks the gap between fixing and shipping to minutes, which changed how fast user-reported bugs (the phone "Load failed") turned into deployed fixes.

SOFTWARE ENGINEERING **Capacity engineering on 512MB.** The memory incident is a compact case study: diagnose (training spikes + resident DataFrames), then cheapest-first remediation — halve numeric dtypes (float64→float32: the data's ~3 significant digits make 7-digit precision free to give up), garbage-collect after heavy endpoints, cap allocator fragmentation — before paying for hardware. The decision ladder (optimize → re-platform → upsize) is standard SRE economics, applied on a student budget.

INDUSTRIAL ENGINEERING **The deployment as a queueing system.** One uvicorn worker serving compute-heavy requests means a long forecast blocks the queue — acceptable at demo traffic, the first bottleneck at scale. Naming the current operating point and its limits (single server, in-memory state, restart-refetch) is itself IE practice: capacity statements should be explicit, not discovered.

13 Deep dive: the frontend stack — languages, technologies, UI/UX design

13.1 The three languages and their division of labour

LANGUAGE	ROLE	WHERE IN THIS PROJECT	JUSTIFICATION
HTML (via JSX)	Structure & semantics — what things <i>are</i>	Everything React renders; <code>index.html</code> shell; <code>role="region"</code> , <code>aria-label</code> on scroll areas	Semantics carry accessibility and machine-readability for free; screen readers and Playwright both navigate by them
CSS	Presentation — what things <i>look like</i> , at every size	<code>styles.css</code> (~1,100 lines) + <code>tokens.css</code> ; custom properties, grid/flex, <code>clamp()</code> , media & container queries	Declarative styling scales: one token change restyles every screen; layout engines (grid/flex) handle reflow no JS could match
JavaScript (ES modules) + JSX	Behaviour — what things <i>do</i>	All components/pages; fetch clients; <code>ResizeObserver</code> ; stream reader	The only language browsers execute; JSX adds compile-time structure so UI code reads as its own output

SOFTWARE ENGINEERING **Separation of concerns across languages.** The trio is itself an architecture: structure, presentation and behaviour evolve at different speeds and are owned by different skills. The project's one deliberate violation — inline styles inside JSX for page-specific layout — was a pragmatic speed trade, and its cost was paid transparently later (the responsive remediation had to target inline grids with attribute selectors). Teaching point: *every violation of separation is a loan; this one's interest payment is documented in `RESPONSIVE_AUDIT.md`.*

13.2 Technology by technology — role and justification

TECHNOLOGY	ROLE	WHY CHOSEN (AND OVER WHAT)
React 18	Component model: <code>UI = f(state)</code> . State changes → React recomputes the affected DOM	Eliminates manual DOM bookkeeping (the largest bug class in vanilla JS apps); component reuse (one KPI card, used everywhere); largest ecosystem/hiring pool. Over Vue/Angular/Svelte: no technical necessity — ecosystem maturity and transferable skill decided it
Vite	Dev server + production bundler; <code>import.meta.env</code> injects build-time config	Near-instant startup and hot reload (native ES modules in dev, esbuild pre-bundling); tree-shaken, minified production build. Over webpack/CRA: order-of-magnitude faster feedback loop; CRA is deprecated in practice
Hash-based SPA navigation	15 lines in <code>App.jsx</code> : <code>state ↔ location.hash (/#supply)</code>	Deep links + back button with zero server configuration (a hash never reaches the server, so the SPA fallback route stays trivial). Over React Router: no nested routes/guards needed yet — a library must earn its dependency
Fetch API + thin clients	<code>client.js</code> normalizes errors and centralizes the base URL; one function per endpoint	The browser's native HTTP; a 30-line wrapper beats axios's 30KB. The two-clients bug (aiClient kept a localhost default → "Load failed" on phones) is the teachable failure: <i>configuration must have exactly one home</i>
ReadableStream reader	<code>aiClient.js</code> reads the AI reply chunk-by-chunk and appends to the chat	Perceived latency: first words appear in ~1s instead of a 20s blank wait — a UX property delivered by a transport choice
Hand-rolled SVG charts	<code>Charts.jsx</code> : 23 chart components drawing primitives (paths, rects, text)	Vector = crisp at any DPI and in print; total design control (serif hero numbers, banded ranges, polarity-aware deltas). Over Recharts/Plotly: no dependency weight or styling fights — the cost, owning responsiveness, was paid once via ResizeObserver
ResizeObserver	Charts measure their <i>own</i> rendered width and draw 1:1, thinning ticks when narrow	Component-level responsiveness: a chart in a half-width card adapts to the card, not the screen — the same philosophy as CSS container queries, applied where drawing happens in JS
localStorage	Remembers sidebar collapsed state across visits	Client-side preference needs no server round-trip and no account
Playwright	112 machine-checked layout assertions at 8 viewports + 52 screenshots	Real-browser truth (Chromium), not JSDOM approximation; the only way "responsive" became a verified claim instead of an opinion

13.3 The UI/UX design system — "Dash": clean, professional, dense, data-first

The design language is named in `styles.css` 's first line. Every visual decision below serves its four adjectives.

Design tokens — the single source of visual truth. ~40 CSS custom properties define the palette, spacing scale (4px baseline), radii, shadows and type ramp; `tokens.css` adds fluid scales (`--step--2...--step-5` , `--space-3xs...--space-3xl`) built with `clamp(rem + vw)` so type and space glide between 320px and 1240px viewports *while still honouring user font-size settings and zoom* (the rem term carries the user preference; a pure-vw scale would trample it — an accessibility subtlety worth teaching).

Color as semantics, not decoration. Two brand hues (steel blue #1e6091 — clinical trust; teal #0d9488 — the action accent) and a strict semantic band shared with the analytics layer: **risk red · watch amber · stable teal · trend blue**. A colour on this UI is a claim about state. Badges use soft tints with dark text rather than saturated fills — higher contrast ratios (WCAG), calmer density. The same four categories drive finding cards, day-card dots and KPI accents, so users learn the code once.

Typography as information hierarchy. Three faces, three jobs: **Inter** (UI sans — tall x-height, legible at 11–13px data densities), **JetBrains Mono** (identifiers and aligned figures), and a **serif display** (Georgia/Times) reserved for hero numbers — the deliberate "boardroom" gesture that makes headline figures feel authoritative. `font-variant-numeric: tabular-nums` wherever digits stack, so columns of numbers align vertically — a small property carrying real scanning speed.

The weather metaphor — the project's central UX bet. Forecasts render as a strip of day cards: big number, likely range, coloured demand dot — deliberately shaped like a weather app. Justification: managers already read weather forecasts daily; the metaphor imports, for free, an intuitive grasp of "a forecast has a range", "further out = less certain", "plan with the range". This is UX theory (recognition over recall, Jakob's Law: users spend most of their time on *other* products) applied to statistical communication.

Plain-language microcopy. The UI says "likely range", "busiest day", "where it should land" — never "95% confidence interval". The accuracy-figure ban in the public app is a governance choice surfaced as copywriting. Rule taught by this project: *the interface speaks operations English; statistics live in the methods chapter*.

Progressive disclosure. Every page answers in three depths: KPI strip (glance) → narrative findings/cards (read) → tables & per-item detail (interrogate). "Read this for me" buttons and the floating assistant add an AI-mediated fourth depth. Nobody meets a dense table before they have seen the headline.

Responsive & accessible mechanics. Off-canvas drawer + hamburger under 820px; fixed inline grids collapse to 1–2 columns; tables become labelled scroll regions (`role="region"` , `tabindex="0"`); touch targets ≥ 44 px (Apple/Android guidance); inputs ≥ 16 px on touch (defeats iOS auto-zoom); `100dvh` instead of `100vh` (mobile URL-bar reality); charts thin their chrome instead of shrinking text. Verified, not asserted — 112 assertions.

The design process itself. Reference mockups first (`project/` , `docs/explore-reference.html`), then componentization, then a written handoff spec (`EXPLORE-HANDOFF.md`), then phased remediation with a stop-for-approval audit. Design ran like engineering: artefacts, contracts, verification.

14 Deep dive: the backend stack & API design

14.1 Why Python

The backend language was decided by the models, not the web layer: SARIMAX, gradient boosting, integer programming and vectorized simulation all live natively in Python's scientific ecosystem. Any other backend language would have meant either reimplementing statistics (unacceptable risk) or running a second service just for the models (unjustified complexity at this scale). Readability was the second argument: Python code doubles as the dissertation's methods documentation.

14.2 Library by library — role and justification

LIBRARY	ROLE	WHY CHOSEN (AND OVER WHAT)
FastAPI	Web framework: decorated functions become endpoints; type hints become validation and docs	ASGI-native (async I/O for GitHub fetches and AI streaming); Pydantic integration gives parse-don't-validate request contracts; automatic OpenAPI at <code>/docs</code> . Over Flask: validation/docs are manual there. Over Django: its ORM/admin/auth weight serves database apps — this app deliberately has no database
Uvicorn	ASGI server: the event loop that accepts sockets and drives the app	The standard FastAPI pairing; async concurrency lets slow upstream I/O (GitHub, Anthropic) overlap without threads
Pydantic	Request/response models (<code>RunRequest</code> , <code>SupplySweepRequest</code> ...)	Invalid input dies at the boundary with a structured 422 — domain code never sees malformed data. "Parse, don't validate" as a framework feature
pandas	Labelled tabular algebra: joins, resampling, rolling windows, group-bys	The lingua franca between ingestion, cleaning, features and models — every layer speaks DataFrame. Its memory appetite is the known cost (hence the float32 diet)
NumPy	Vectorized arrays beneath everything; the simulation's 800×90 draws in single calls	C-speed loops; the reason Monte-Carlo over 30 items × 800 reps finishes in seconds. Teaching point: <i>in scientific Python, the loop you don't write is the optimization</i>
statsmodels	SARIMAX estimation with inference: AIC, prediction intervals	The inference-grade time-series library — scikit-learn has no ARIMA and no principled intervals. Chosen where <i>uncertainty quantification</i> is the deliverable
scikit-learn	<code>GradientBoostingRegressor</code> + metrics	Uniform estimator API (fit/predict) makes models swappable; battle-tested boosting without the operational weight of XGBoost/LightGBM at this data size
PuLP + CBC	Algebraic modelling of the roster MILP; CBC solves by branch-and-bound	Write the <i>mathematics</i> (variables, constraints, objective), not the algorithm; CBC is free/open-source and exact at this problem size. Over commercial solvers: nothing here needs Gurobi's scale. Over heuristics: exactness is affordable, so approximation is unjustified
httpx	Async HTTP client: GitHub Contents API/raw CDN, self-calls from AI tools	Async + streaming + granular timeouts in one modern API (requests is sync-only)
truststore + certifi	TLS trust via the OS certificate store, with a bundled fallback	Survives corporate TLS-inspecting networks (a failure mode this very project's network exhibits — plain curl fails where the app succeeds)
python-dotenv	Loads <code>api/.env</code> into the environment; <code>data_source</code> re-reads per call	Twelve-Factor config with zero infrastructure; the re-read means a token fix needs no restart
orjson	Rust-speed JSON with native NumPy serialization (custom response class)	The enabler of the memory diet: float32/int32 arrays serialize everywhere without per-endpoint casting — one response class instead of a hundred <code>float()</code> calls
anthropic SDK	Typed access to Claude: messages, streaming, tool definitions	Official SDK handles the tool-use protocol and streaming framing; the app keeps its own loop for auditability
pulp's CBC / gc / MALLOC_ARENA_MAX	Operational hygiene: collect after heavy endpoints; cap allocator arenas	Capacity engineering: the cheapest fixes for a 512MB ceiling, applied before paying for hardware

14.3 API design — the principles the endpoints obey

Resource-oriented URLs, verb-carrying methods. Prefixes are nouns (`/api/forecast` , `/api/supply`); HTTP methods carry intent: **GET** for reads that are safe to repeat and cache (`/overview` , `/last`), **POST** for computations with bodies and side effects (running a forecast trains a model and updates the last-run cache — not idempotent, hence POST). Teaching rule: *the method is a contract about consequences, not a syntax choice*.

Status codes as a vocabulary. The API distinguishes its failures: **422** malformed request (Pydantic, automatic); **400** semantically invalid (horizon not in the allowed set); **409** valid request, wrong state ("G1 not merged — build it first": the client can fix this by acting, which is exactly what 409 Conflict means); **401/403** upstream auth (the GitHub token saga); **503** dependency missing (handover artefacts absent → the error message even contains the fix command); **504** upstream exhausted retries. The UI maps each to a different user message — the codes are load-bearing.

Errors that teach. `{"error": "g1_not_merged", "message": "...Build it on the Prepare page first."}` — a machine-readable code plus a human sentence naming the remedy. The submodule error going as far as including `git submodule update --init --recursive` paid off on the day it was hit.

The materialization pattern (`/last`). Stochastic computations (retraining forecasts, optimization runs) are cached server-side after each run; every consumer — the page, the AI, the Action Center — reads the same materialized result. This one pattern eliminated an entire class of "the AI says 63 but the screen says 66" contradictions. General form: *compute once, reference everywhere; never let two consumers recompute a nondeterministic result independently*.

Streaming where latency is felt. `/api/ai/*` endpoints return chunked plain text, not JSON — the browser renders words as they are generated. Different payload disciplines for different UX needs, inside one API.

Thin routers, fat core. Routers own HTTP concerns only (validation, status codes, serialization); domain logic lives in `api/core` and is imported as plain functions. Consequence: the forecasting engine is testable without a web server, and the AI's tools call the same core the pages do. This is hexagonal architecture in miniature — ports (routers) around a domain (core) — without the ceremony.

Cross-cutting concerns via middleware. CORS headers and the post-request `gc.collect()` for heavy paths are middleware — policies applied uniformly in one place rather than remembered in forty endpoints.

The self-documenting contract. FastAPI generates OpenAPI from the code itself: `/docs` is a living, always-current API reference that doubles as a manual test console — the reason "open /docs" is this project's first health check.

15 The decision register

Every consequential choice, its rejected alternatives, and the justification — the oral-exam table.

DECISION	REJECTED ALTERNATIVES	JUSTIFICATION
FastAPI backend	Django, Flask, Node	Python-native scientific stack; typed validation; auto docs; async
React + Vite SPA, hash navigation	Next.js, router library	No SEO need; 15-line router suffices; fewer moving parts to learn/maintain
Data in private GitHub repo, fetched by token	Bundled in app repo; database	Privacy separation; versioned provenance; zero infra at this scale
In-memory registries, no persistence	PostgreSQL	Health-data privacy posture; adequate for single-process demo; revisit for multi-user
SARIMAX + Gradient Boosting, both	One model; deep learning	Champion–challenger falsifiability; DL unjustified at ~2400 daily points
Recursive multi-step	Direct per-horizon models	One model, honest widening intervals; horizon short ($\leq 30d$)
Empirical residual intervals (ML)	Quantile regression, conformal	Assumption-light, calibrated by construction on validation errors; simpler to defend
Rolling-origin backtest UI	Internal metrics only	Users audit accuracy themselves — trust through verification
Role-based plug-in analytics	Hand-coded dashboard	Open–closed extension; portable to any dataset; failure isolation
Integer programming (PuLP/CBC) for rostering	Greedy/manual heuristics	Provable legality & optimality; problem size solvable exactly in seconds
(s,S) with simulation grid-search	EOQ; fixed schedules; closed-form only	Stochastic demand; interacting costs intractable analytically; policy executable by staff
Negative binomial demand in simulation	Poisson	Observed over-dispersion; Poisson understates tails → under-provisioning
Common random numbers across policy arms	Independent runs	Variance reduction: differences reflect policy, not luck
Read-only AI tools + audit log + prompt policy	Free-form AI; write-capable agent	Grounding, least privilege, inspectability — governance by architecture
Keyword retrieval over 12 cards	Vector DB + embeddings	Simplest mechanism meeting the need; upgrade path known
Hand-rolled SVG charts	Recharts/Plotly	Exact design control, no dependency weight; cost was owning responsiveness (paid later, once)
Docker single-service (API serves frontend)	Separate frontend hosting	Same-origin (no CORS), one deploy, one URL — fewer failure modes
Render Starter + memory diet	Immediate upscale; Railway	Cheapest-first ladder; migration path preserved and priced
Playwright E2E assertions before claiming "responsive"	Manual checking	Claims require machine-checkable evidence — 112 assertions, 8 widths

16 A training curriculum — six sessions to teach this system

#	SESSION (≈2H)	COVERS	HANDS-ON EXERCISE
1	The forecast-to-decision idea + data layer	Ch 1–2	Clone the repo, create a scoped PAT, run backend + frontend locally, break and fix the token
2	Preparation & leakage	Ch 3	Build G1 on the Prepare page; introduce a deliberate leak (shuffle-CV a lagged feature) and watch accuracy lie
3	The two forecasting engines & validation	Ch 4–5	Backtest both engines on three past dates; explain each interval; find a date where the "worse" model wins
4	Optimization: roster IP + (s,S)	Ch 7–9	Change one constraint (44h cap) and re-solve; move service level 90→98% and chart the cost curve
5	The governed AI layer	Ch 10	Add a 13th knowledge card and a new read-only tool; try to make the assistant invent a number — then explain why it wouldn't
6	Architecture, shipping, and the decision register	Ch 6, 11–15	Write a new Explore analyzer (one file); deploy it via push; defend two register rows in a mock viva

Assessment prompts (use in any session)

- "Walk the chain: how does a 12.6% MAPE become a Rand amount of saved safety stock?" (Ch 8's σ argument.)
- "Your accuracy doubled overnight. What do you suspect first, and how do you check?" (Leakage; rolling-origin backtest.)
- "Why is the AI allowed to read the roster but not write it — and where in the code is that enforced?" (Tool definitions; least privilege by construction.)
- "Management wants 99.9% service level on all items. Argue back with the four-cost taxonomy."
- "Which single file would you change to point the whole analytics layer at a fisheries dataset?" (The GroupProfile.)